Python

Day-wise Notes

Contents

Day 32 · Methods in sets 2

BASIC METHODS . . . 2

symmetric_difference . . . 2

difference . . . 2

special methods: . . . 3

Day 33 · dictionary 4

Day 34 · dictionary methods 5

Day 35 · for loop with else 6

Day 36 · exception handling 7

Day 37 · finally keyowrd 8

Day 38 · raise error 9

Day 41 · short hand if else 10

use this as conditional statements . . . 10

Day 42 · enumerate function 11

DAY 32

Methods in sets

```
CODE
s1={1,2,3,4,7}
s2={4,5,6}
```

BASIC METHODS

union : contains all the elements of both sets

```
CODE
union =s1.union(s2)

update = s1.update(s2)# now s1 set contains all the value there in both set


print(union)

s3={1,2,3,4,57}
intersection =s3.intersection(s2)

print(intersection)

intersection_update= s2.intersection_update(s1)
# print(intersection_update)-----> output none because same elements that is the proof btw if you think it in a way
```

symmetric_difference

symmetric_difference : values that are not common in the sets <-----values of both sets come-------->

```
CODE
symmetric_difference =s1.symmetric_difference(s2)
symmetric_difference_update =s1.symmetric_difference_update(s2)
```

difference

difference : removes the common elements from the dominating set and gives other elements

```
CODE
difference =s1.difference(s2)
difference_update =s1.difference_update(s2)
```

special methods:

isdisjoint(): false i elements are there common, true if no elements are in common

issuperset(): all elements of of s2 are there in s1

issubset(): elemets in s2 are all there in in s1

add():

update()

remov0():it raises error /discard():it makes program in flow do n't show error

pop(): randomly removes element from the set i.e. last element

del cities : delets set

clear(): just clears the set

DAY 33 dictionary

dictionary: ordered key value pairs

CODE

```
dict= {
    "raja":12,#"raja"-----> key, 12----> value
    "dh":43,
    "ttt":4332
}
print(dict["dh"])
#loop to access or selected value using range function
for i in dict:
    print(dict[i])

print(dict.items())
```

in f strings you can use this as :

CODE

```
for key , value in dict.items():

    print(f"Name :{key}, \nmarks: {value}")

# using two variable in f string via dictionary defined
```

DAY 34

dictionary methods

CODE

```
employee1={
    23:45,
    24:88

}

manager={
    25:55
}

employee1.update(manager)#update the dictionary

print(employee1)

manager.clear()#clear the dictionary

employee1.pop(24)#removes the selected key value
print(employee1)

employee1.popitem()#removes the last key value pair

print(employee1)

# del employee1 <-----deletes the dictionary------->
```

DAY 35

for loop with else

we can use else with for loop

CODE

```
for i in range(5):
    print(i)

else:
    print("Sorry no i")

    #loop is completed till the end of the loop if loop breaks then only this balue is not printed


for i in range(6):
    print(i)
    if i==5:
        break

else :
    print("sorry ")
```

DAY 36 exception handling

exception handling: handles the error and makes the further code run the same way as they should go with the flow

CODE

```
a = int(input("Enter Your Number:"))

for i in range(1, 11):
    print(f"{a} X {i} = {a*i}")

# if input in a is not an integer then this shows error
```

CODE

```
try:
    n = int(input("Number:"))
    for i in range(1, 11):
        print(f"{n} X {i} = {n*i}")

except ValueError:
    print("Invalid Input")


except IndexError:
    print("Index error")


a = input("Hello Your Name Please:")
print(a)
```

CODE

```
"""
If the user enters something that's not a number (like "abc") → catch it and print "Please enter
valid numbers."
If the user enters 0 as the second number (division by zero) → catch it and print "Cannot divide
by zero."
If neither error occurs → print the division result.
"""
```

DAY 37

finally keyowrd

finally keyowrd : a program that is always excecuted

CODE

```
try:
    a = int(input("Enter Your Number:"))
except:
    print("Invalid Input")

finally:
    print("Your Desired Output")


print(
    "Your desired output"
)  # this is also always printed except when this is in function and function works as a return statement
```

finally is always excecuted

CODE

```
def random_print():
    try:
        a = int(input("Enter Your Number:"))
        return 1
    except:
        print("Invalid Input")
        return 0
    finally:
        print("Your Desired Output")


    print(
        "Your desired output"
    )  #this value will never be excecuted but if we use finally it will be excecuted
```

DAY 38

raise error

raise error : whenever required error to be there as a custom

CODE

```
a = int(input("Enter Your Number that should be less that 10:"))

if a >= 10:
    raise ValueError("Invalid input \n Enter Your Number Below 10")

b = input("Number b/w 5&9:")
if b == "quit":
    print("Yeahhh")
elif not b.isdigit():
    raise ValueError("Not An Integer")
else:
    b=int(b)
    if (b > 5) and (b < 9):
        print(b)
    else:
        raise ValueError
```

DAY 41

short hand if else

must use this for only short singlt hand statements not for more than one statement

CODE

```
a= 45
b=884

print("A") if a>b else print("Equal") if a==b else print("B")
# <---1--><-2---><------------3-------------><------4------>
```

1==> statement to be excecuted 2==> if + condition behind it 3==>else + statement + if + condition

also can be used a sa conditionals

use this as conditional statements

CODE

```
c="fuck" if a<b else ""# double quote returns nothing
# if == true ----> c is printed and if not else statement is printed

print(c)
```

Question: Write a Python program that checks if a number is positive, negative, or zero using the shorthand if-else (ternary operator). Store the result in a variable called result and print it.

CODE

```
num = int(input("Enter Your Number:"))

# result = if num>0 print("Positive") else (num==0) print("Zero") else print("negative")
# print(result)

result = print("positive") if num>0 else print("Zero") if num==0 else print("Negative ")
print(result)
```

DAY 42

enumerate function

In built python function that allows you to access the index as well the value at the same time

CODE
```
lst= [1,2,3,4,5,6,7,8,9]

index =1
for i in lst:
    print(i)
    if index ==3:
        print("this is required ")
    index+=1
```

this can be easily done by python

CODE
```
for index , i in enumerate(lst):
    print(index , ":",i)#this gives index : value key value order pair

    # as well index os showing this can be changed
```

start index:

CODE
```
for index , i in enumerate(lst, start =4):#now when index =4 it starts from there
    print(index , ":",i)#this gives index : value key value order pair
```